



**SCHOOL OF ENGINEERING AND NATURAL
SCIENCES**

EEE

Embedded Systems

Project

Name, Surname: Mohammed SharafAldeen

Department, Student ID: EEE,61220027

Date: 6/9/2026

Introduction

Floating-point numbers are commonly used in digital systems to represent values that contain fractions or very large and small numbers. In this project, a simplified 8-bit floating-point format, called FP8, was used. This format consists of one sign bit, four exponent bits, and three mantissa bits. The exponent uses a bias value of 7. Although FP8 has a smaller range and lower precision than standard floating-point formats, it is useful for learning how floating-point arithmetic works in hardware.

The main objective of this project was to design a Verilog HDL module that adds two FP8 floating-point numbers. The design had to support normal numbers, subnormal numbers, signed values, zero, infinity, and NaN cases. Special cases such as infinity and NaN were handled according to the rules given in the project description. For normal addition, the FP8 inputs were converted into an integer representation, added together, and then converted back into FP8 format with rounding.

The project also required exhaustive verification using a testbench. Since each FP8 input is 8 bits, there are 256 possible values for each input. Therefore, the testbench checked all 65,536 possible input combinations and compared the output with a reference memory file. After simulation, the design was implemented on the DE10-Lite FPGA board. Switches were used to enter the FP8 input values, pushbuttons were used to store the inputs, and the HEX displays were used to show the selected value or result as a readable floating-point number.

This report explains the FP8 format, the design method used for the adder, the testbench verification, and the FPGA implementation results.

Verilog Codes:

```
1 module fp8_adder (A, B, SUM);
2
3 input  [7:0] A;
4 input  [7:0] B;
5 output reg [7:0] SUM;
6
7 // split A and B
8 wire A_s, B_s;
9 wire [3:0] A_e, B_e;
10 wire [2:0] A_m, B_m;
11
12 assign A_s = A[7];
13 assign A_e = A[6:3];
14 assign A_m = A[2:0];
15
16 assign B_s = B[7];
17 assign B_e = B[6:3];
18 assign B_m = B[2:0];
19
20 // special cases
21 wire A_is_nan, B_is_nan;
22 wire A_is_inf, B_is_inf;
23 wire A_is_zero, B_is_zero;
24
25 assign A_is_nan = (A_e == 4'b1111) && (A_m != 3'b000);
26 assign B_is_nan = (B_e == 4'b1111) && (B_m != 3'b000);
27
28 assign A_is_inf = (A_e == 4'b1111) && (A_m == 3'b000);
29 assign B_is_inf = (B_e == 4'b1111) && (B_m == 3'b000);
30
31 assign A_is_zero = (A_e == 4'b0000) && (A_m == 3'b000);
32 assign B_is_zero = (B_e == 4'b0000) && (B_m == 3'b000);
33
34 // change FP8 to integer first, then add
35 wire signed [18:0] A_num;
36 wire signed [18:0] B_num;
37 wire signed [18:0] sum_num;
```

```

40 assign B_num = to_number(B);
41
42 assign sum_num = A_num + B_num;
43
44 always @(*) begin
45     if (A_is_nan || B_is_nan)
46         SUM = 8'b01111001;           // NaN
47     else if (A_is_inf && B_is_inf && (A_s != B_s))
48         SUM = 8'b01111001;           // +inf + -inf
49     else if (A_is_inf)
50         SUM = {A_s, 4'b1111, 3'b000}; // A is infinity
51     else if (B_is_inf)
52         SUM = {B_s, 4'b1111, 3'b000}; // B is infinity
53     else if (sum_num == 0) begin
54         if (A_is_zero && B_is_zero && A_s && B_s)
55             SUM = 8'b10000000;       // -0
56         else
57             SUM = 8'b00000000;       // +0
58     end
59     else
60         SUM = to_fp8(sum_num);
61 end
62
63 // This function changes FP8 into an integer. In this code, 512 means 1.0.
64 function signed [18:0] to_number;
65     input [7:0] x;
66
67     reg x_s;
68     reg [3:0] x_e;
69     reg [2:0] x_m;
70     reg [18:0] size;
71
72     begin
73         x_s = x[7];
74         x_e = x[6:3];
75         x_m = x[2:0];
76
77         if (x_e == 4'b0000)
78             size = {16'b0, x_m};
79         else
80             size = {15'b0, 1'b1, x_m} << (x_e - 1);
81
82         if (x_s)
83             to_number = -size;
84         else
85             to_number = size;
86     end
87 endfunction
88
89 // This function changes the integer answer back to FP8.
90 function [7:0] to_fp8;
91     input signed [18:0] value;
92
93     reg ans_s;
94     reg [18:0] size;
95     reg [3:0] ans_e;
96     reg [4:0] ans_m;
97
98     reg [18:0] remainder;
99     reg [18:0] half;
100     integer shift;
101
102     begin
103         // get sign and positive size
104         if (value[18]) begin
105             ans_s = 1'b1;
106             size = -value;
107         end
108         else begin
109             ans_s = 1'b0;
110             size = value;
111         end
112
113         // too big, so make it infinity
114         if (size >= 19'd126976) begin
115             to_fp8 = {ans_s, 4'b1111, 3'b000};
116         end
117
118         // very small number
119         else if (size < 19'd8) begin
120             to_fp8 = {ans_s, 4'b0000, size[2:0]};
121         end
122
123         // normal number
124         else begin
125

```

```

136         if (size < 19'd16)      ans_e = 4'd1;
137     else if (size < 19'd32)      ans_e = 4'd2;
138     else if (size < 19'd64)      ans_e = 4'd3;
139     else if (size < 19'd128)     ans_e = 4'd4;
140     else if (size < 19'd256)     ans_e = 4'd5;
141     else if (size < 19'd512)     ans_e = 4'd6;
142     else if (size < 19'd1024)    ans_e = 4'd7;
143     else if (size < 19'd2048)    ans_e = 4'd8;
144     else if (size < 19'd4096)    ans_e = 4'd9;
145     else if (size < 19'd8192)    ans_e = 4'd10;
146     else if (size < 19'd16384)   ans_e = 4'd11;
147     else if (size < 19'd32768)   ans_e = 4'd12;
148     else if (size < 19'd65536)   ans_e = 4'd13;
149     else                          ans_e = 4'd14;
150
151     shift = ans_e - 1;
152     ans_m = size >> shift;
153
154     // check if we need to round up
155     if (shift > 0) begin
156         remainder = size - ({14'b0, ans_m} << shift);
157         half = 19'd1 << (shift - 1);
158
159         if ((remainder > half) || ((remainder == half) && ans_m[0]))
160             ans_m = ans_m + 1'b1;
161     end
162
163     // if mantissa became too big
164     if (ans_m >= 5'd16) begin
165         ans_m = 5'd8;
166         ans_e = ans_e + 1'b1;
167     end
168
169     // exponent too big means infinity
170     if (ans_e >= 4'd15)
171         to_fp8 = {ans_s, 4'b1111, 3'b000};
172     else
173         to_fp8 = {ans_s, ans_e, ans_m[2:0]};
174     end
175 end
176 endfunction
177
178 endmodule

```

Figure 1. Verilog HDL Code for the FP8 Floating-Point Adder

This code implements the main `fp8_adder` module. It separates each 8-bit FP8 input into sign, exponent, and mantissa fields, then checks special cases such as NaN, infinity, and zero. For finite inputs, the code converts each FP8 value into an integer form where 512 units represent 1.0. The two integer values are then added, and the result is converted back to FP8. During this conversion, the result is rounded to the nearest representable FP8 value. If the result is too large, the code outputs infinity. If the result is very small and still nonzero, it can be represented using the subnormal FP8 format.

```

1 module tb_fp8_adder;
2
3 reg [7:0] A;
4 reg [7:0] B;
5 wire [7:0] SUM;
6
7 reg [7:0] expected_answer [0:65535];
8
9 integer a;
10 integer b;
11 integer wrong_count;
12
13 fp8_adder uut (A, B, SUM);
14
15 initial begin
16     wrong_count = 0;
17
18     // read the correct answers from the memory file
19     $readmemh("fp8_ref.mem", expected_answer);
20
21     // test all possible values of A and B
22     for (a = 0; a < 256; a = a + 1) begin
23         for (b = 0; b < 256; b = b + 1) begin
24             A = a;
25             B = b;
26
27             #10;
28
29             if (SUM != expected_answer[a*256 + b]) begin
30                 wrong_count = wrong_count + 1;
31                 $display("Wrong answer: A=%h B=%h SUM=%h Expected=%h",
32                     A, B, SUM, expected_answer[a*256 + b]);
33             end
34         end
35     end
36
37     $display("Test finished");
38     $display("Total tests = 65536");
39     $display("Wrong tests = %0d", wrong_count);
40     $display("Correct tests = %0d", 65536 - wrong_count);
41
42     $stop;
43 end
44
45 endmodule

```

Figure 2. Verilog Testbench Code for Exhaustive FP8 Adder Verification

This testbench verifies the fp8_adder module by applying every possible input combination for A and B. Since each input is 8 bits, each one has 256 possible values, so the testbench checks $256 \times 256 = 65,536$ total cases. The correct answers are loaded from the fp8_ref.mem file using \$readmemh. For each test case, the output SUM is compared with the expected value. If the result is different, the error counter increases and the wrong case is printed.

```

VSIM 12> run -all
# Test finished
# Total tests = 65536
# Wrong tests = 0
# Correct tests = 65536
# ** Note: $stop      : tb_fp8_adder.v(46)
#   Time: 655360 ps  Iteration: 0  Instance: /tb_fp8_adder
# Break in Module tb_fp8_adder at tb_fp8_adder.v line 46

```

Figure 3. Simulation Result for FP8 Adder Test

The ModelSim simulation result shows that the testbench completed all 65,536 input combinations. The final output reports Wrong tests = 0 and Correct tests = 65536, which means the FP8 adder

passed all possible test cases successfully. This confirms that the Verilog adder produces the same results as the reference file for every possible pair of FP8 inputs.

```
1 module fp8_value_display (X, HEX5, HEX4, HEX3, HEX2, HEX1, HEX0);
2
3 input [7:0] X;
4
5 output reg [7:0] HEX5, HEX4, HEX3, HEX2, HEX1, HEX0;
6
7 // split the FP8 number
8 wire X_s;
9 wire [3:0] X_e;
10 wire [2:0] X_m;
11
12 assign X_s = X[7];
13 assign X_e = X[6:3];
14 assign X_m = X[2:0];
15
16 // check special cases
17 wire X_is_nan;
18 wire X_is_inf;
19
20 assign X_is_nan = (X_e == 4'b1111) && (X_m != 3'b000);
21 assign X_is_inf = (X_e == 4'b1111) && (X_m == 3'b000);
22
23 reg [18:0] size;
24
25 reg [9:0] whole_num;
26 reg [6:0] decimal_part;
27
28 reg [3:0] h, t, o, d1, d2;
29
30 integer n;
31
32 always @(*) begin
33
34     HEX5 = 8'b11111111;
35     HEX4 = 8'b11111111;
36     HEX3 = 8'b11111111;
37     HEX2 = 8'b11111111;
38     HEX1 = 8'b11111111;
39     HEX0 = 8'b11111111;
40
41     size = 19'd0;
42     whole_num = 10'd0;
43     decimal_part = 7'd0;
44
45     h = 4'd0;
46     t = 4'd0;
47     o = 4'd0;
48     d1 = 4'd0;
49     d2 = 4'd0;
50
51     n = 0;
52
53     // show nAn
54     if (X_is_nan) begin
55         HEX5 = 8'b11111111;
56         HEX4 = 8'b10101011; // n
57         HEX3 = 8'b10001000; // A
58         HEX2 = 8'b10101011; // n
59         HEX1 = 8'b11111111;
60         HEX0 = 8'b11111111;
61     end
62 end
```

```

63 // show InF or -InF
64 else if (X_is_inf) begin
65
66     if (X_s)
67         HEX5 = 8'b10111111; // -
68     else
69         HEX5 = 8'b11111111;
70
71     HEX4 = 8'b11111001; // I
72     HEX3 = 8'b10101011; // n
73     HEX2 = 8'b10001110; // F
74     HEX1 = 8'b11111111;
75     HEX0 = 8'b11111111;
76 end
77
78 // normal number, subnormal number, or zero
79 else begin
80
81     if (X_s)
82         HEX5 = 8'b10111111; // -
83     else
84         HEX5 = 8'b11111111;
85
86     // convert FP8 to integer units, 512 units means 1.0
87     if (X_e == 4'b0000)
88         size = {16'b0, X_m};
89     else
90         size = ({15'b0, 1'b1, X_m} << (X_e - 1));
91
92     // number before decimal point
93     whole_num = size / 512;
94
95     // two digits after decimal point
96     decimal_part = ((size - whole_num * 512) * 100) / 512;
97
98     d1 = decimal_part / 10;
99     d2 = decimal_part - d1 * 10;
100
101     n = whole_num;
102     h = n / 100;
103     n = n - h * 100;
104     t = n / 10;
105     o = n - t * 10;
106
107     HEX4 = show_digit(h); // hundreds
108     HEX3 = show_digit(t); // tens
109     HEX2 = show_digit(o); // ones
110     HEX1 = show_digit(d1); // first decimal value
111     HEX0 = show_digit(d2); // second decimal value
112
113     // decimal point after ones digit
114     HEX2[7] = 1'b0;
115 end
116 end
117
118 function [7:0] show_digit;
119 input [3:0] num;
120 begin
121     case (num)
122         4'd0: show_digit = 8'b11000000;
123         4'd1: show_digit = 8'b11111001;
124         4'd2: show_digit = 8'b10100100;
125         4'd3: show_digit = 8'b10110000;
126         4'd4: show_digit = 8'b10011001;
127         4'd5: show_digit = 8'b10010010;
128         4'd6: show_digit = 8'b10000010;
129         4'd7: show_digit = 8'b11111000;
130         4'd8: show_digit = 8'b10000000;
131         4'd9: show_digit = 8'b10010000;
132         default: show_digit = 8'b11111111;
133     endcase
134 end
135 endfunction
136
137 endmodule

```

Figure 4. Verilog HDL Code for the FP8 Value Display Module

This code implements the `fp8_value_display` module, which converts one 8-bit FP8 value into a readable floating-point display on the six HEX displays. It first separates the FP8 number into sign, exponent, and mantissa fields, then checks special cases such as NaN and infinity. If the value is NaN, it displays nAn; if it is infinity, it displays INF or -INF. For normal, subnormal, and zero

values, the code converts the FP8 value into integer units where 512 units represent 1.0. It then separates the number into hundreds, tens, ones, and two decimal digits. HEX5 shows the negative sign if needed, while HEX4 to HEX0 show the floating-point value in the form 000.00. The decimal point is turned on after the ones digit.

```

1 module fp8_top (fr_SW, fr_KEY, to_LEDR, to_HEX0, to_HEX1, to_HEX2, to_HEX3, to_HEX4, to_
2 input [9:0] fr_SW;
3 input [1:0] fr_KEY;
4 output [9:0] to_LEDR;
5 output [7:0] to_HEX0, to_HEX1, to_HEX2, to_HEX3, to_HEX4, to_HEX5;
6
7 reg [7:0] A;
8 reg [7:0] B;
9
10 wire [7:0] SUM;
11 reg [7:0] display_value;
12
13 assign to_LEDR = fr_SW;
14
15
16 always @(negedge fr_KEY[0]) begin
17     A <= fr_SW[7:0];
18 end
19
20 always @(negedge fr_KEY[1]) begin
21     B <= fr_SW[7:0];
22 end
23
24 fp8_adder u1 (A,B,SUM);
25
26 always @(*) begin
27     case (fr_SW[9:8])
28         2'b00: display_value = fr_SW[7:0];
29         2'b01: display_value = A;
30         2'b10: display_value = B;
31         2'b11: display_value = SUM;
32         default: display_value = fr_SW[7:0];
33     endcase
34 end
35
36 fp8_value_display d1 (display_value, to_HEX5, to_HEX4, to_HEX3, to_HEX2, to_HEX1, to_HEX0);
37
38 endmodule

```

Figure 5. Verilog HDL Top Module for DE10-Lite FPGA Implementation

This code implements the `fp8_top` module, which connects the FP8 adder design to the DE10-Lite FPGA board. Switches SW7–SW0 are used to enter an 8-bit FP8 value. Pressing KEY0 stores the switch value as input A, and pressing KEY1 stores the switch value as input B. The `fp8_adder` module calculates the sum of A and B. Switches SW9–SW8 are used to choose what is shown on the HEX displays: the current switch value, stored A, stored B, or the final SUM. The selected value is sent to the `fp8_value_display` module so it can be displayed as a floating-point number on the six HEX displays.

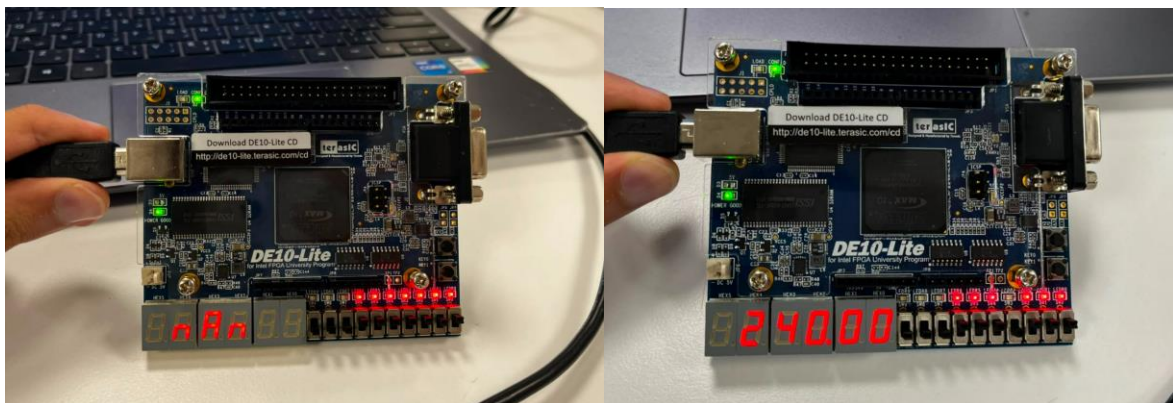




Figure 6. FPGA Test 1: NaN Input Special Case

This test verifies the NaN handling rule of the FP8 adder. One input is the finite FP8 value 240.00, and the other input is a NaN value. According to the project special-case table, if one of the inputs is NaN, the output must also be NaN. The FPGA result displays nAn, confirming that the adder correctly detects a NaN input and returns NaN as required.

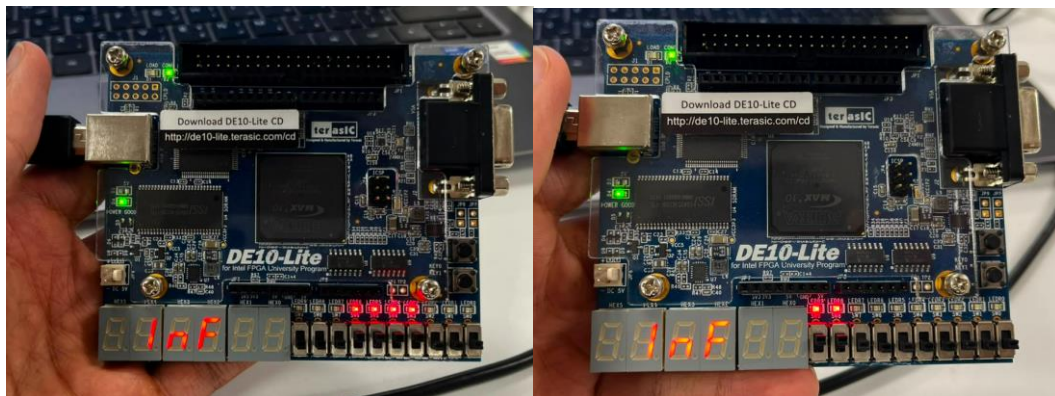


Figure 7. FPGA Test 2: Infinity Plus Infinity Special Case

This test verifies the infinity handling rule of the FP8 adder when both inputs are infinity with the same sign. According to the project requirements, $(+\text{infinity}) + (+\text{infinity})$ must return $+\text{infinity}$. The FPGA output displays INF, confirming that the adder correctly detects the special case and produces the required result.



Figure 8. FPGA Test 3: Negative Infinity Plus Finite Value

This test verifies the special-case rule for adding negative infinity with a finite FP8 number. According to the project requirements, $(-\text{infinity}) + (\text{finite value})$ must return $-\text{infinity}$. The FPGA output displays $-\text{INF}$, confirming that the adder correctly keeps the result as negative infinity when one input is $-\text{INF}$ and the other input is a normal finite value.

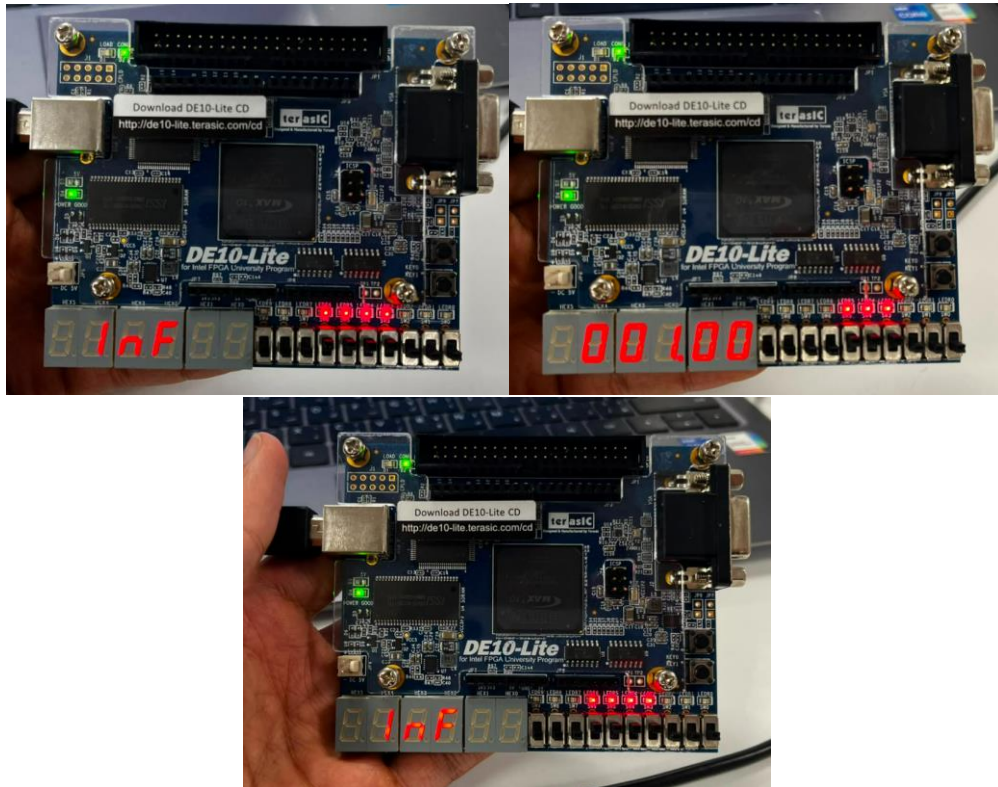


Figure 9. FPGA Test 4: Positive Infinity Plus Finite Value

This test verifies the special-case rule for adding positive infinity with a finite FP8 number. The first input is $+\text{INF}$, and the second input is a finite value, shown as 001.00. According to the project requirements, $(+\text{infinity}) + (\text{finite value})$ must return $+\text{infinity}$. The FPGA output displays INF , confirming that the adder correctly handles this special case.

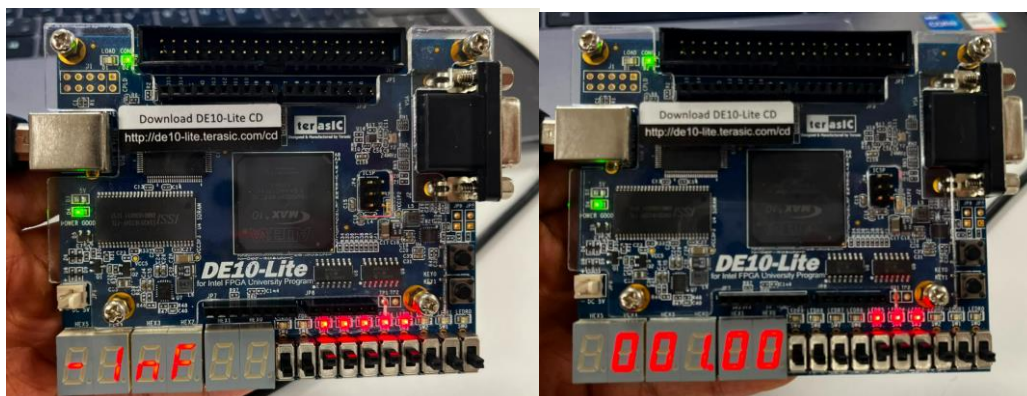




Figure 10. FPGA Test 5: Negative Infinity Plus Finite Value

This test verifies the FP8 special-case rule for adding negative infinity with a finite number. The first input is $-\text{INF}$, and the second input is a finite value shown as 001.00. According to the project requirements, $(-\text{infinity}) + (\text{finite value})$ must return $-\text{infinity}$. The FPGA output displays $-\text{INF}$, confirming that the adder handles this case correctly.

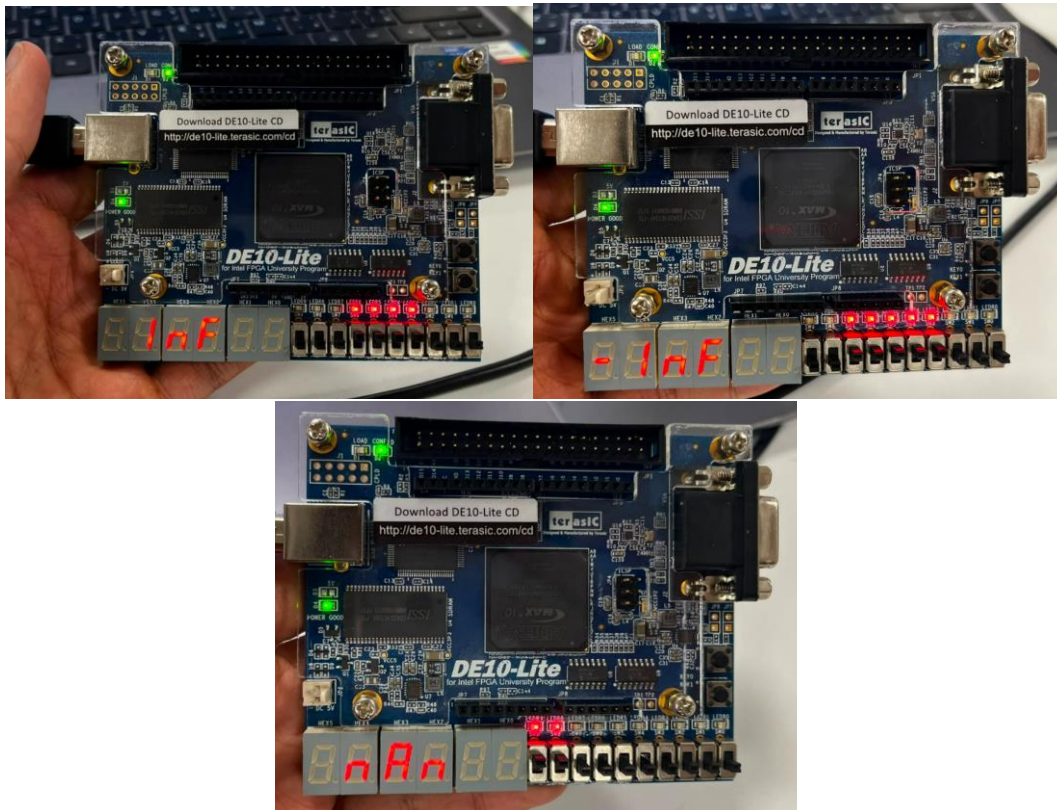


Figure 11. FPGA Test 6: Positive Infinity Plus Negative Infinity

This test verifies the FP8 special-case condition when the two inputs are infinities with opposite signs. The first input is $+\text{INF}$, the second input is $-\text{INF}$, and according to the project requirements, $(+\text{infinity}) + (-\text{infinity})$ must return NaN. The FPGA result displays NaN, so the implemented adder handles this special case correctly.

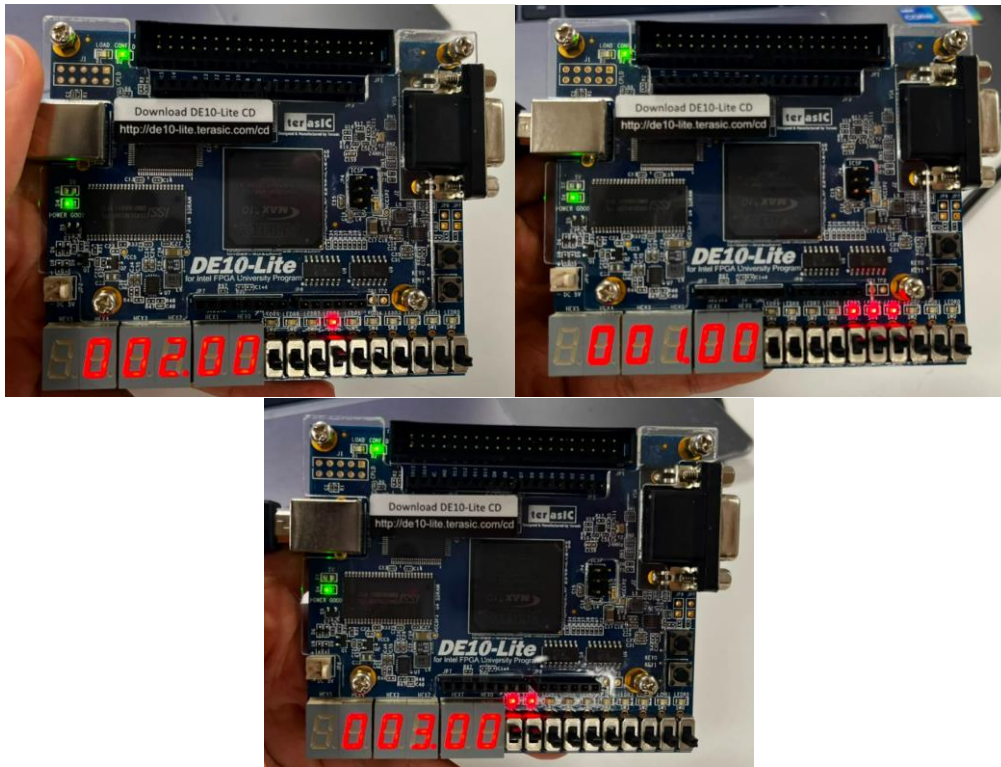


Figure 12. FPGA Addition Test: $2.00 + 1.00$

This test verifies the normal FP8 addition operation using two finite positive numbers. The first input is 002.00, and the second input is 001.00. The expected result is 003.00. The FPGA output displays 003.00, confirming that the FP8 adder correctly adds normal finite values and displays the result on the DE10-Lite HEX displays.

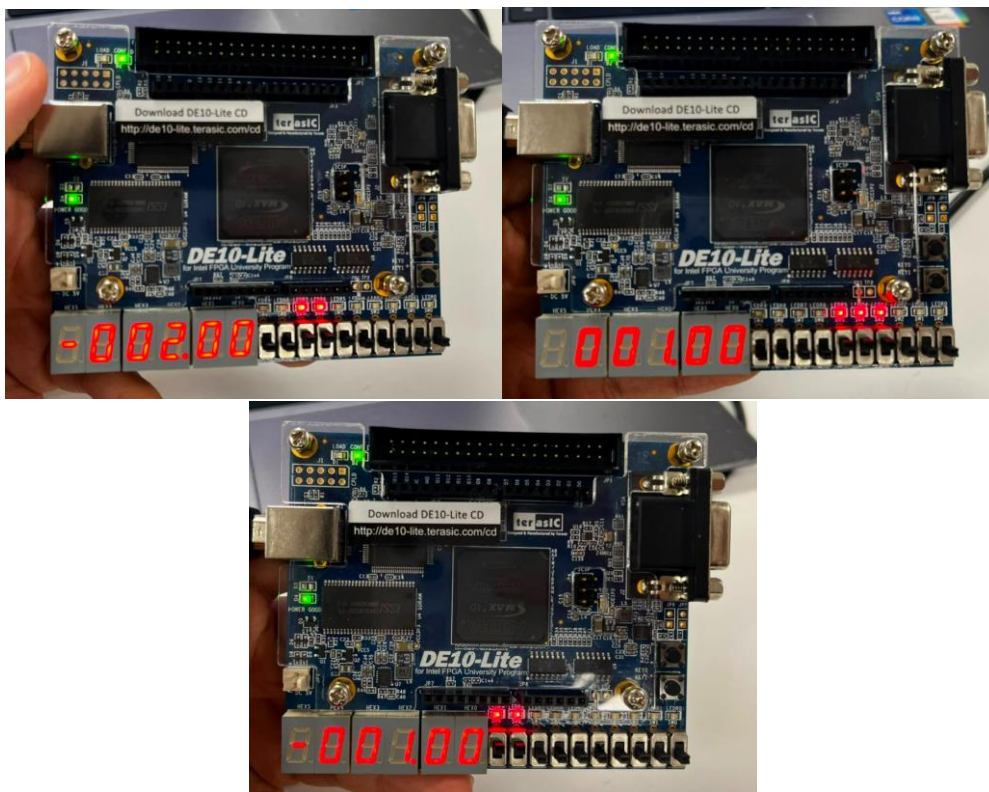


Figure 13. FPGA Addition Test: Negative Result

This test verifies that the FP8 adder correctly handles addition when the result is negative. The first input is -002.00, and the second input is 001.00. The expected result is -001.00. The FPGA output displays -001.00, confirming that the adder correctly performs signed FP8 addition and that the display module correctly shows the negative sign on HEX5.

Conclusion

In this project, an 8-bit FP8 floating-point adder was designed, simulated, and implemented on the DE10-Lite FPGA board using Verilog HDL. The FP8 format used one sign bit, four exponent bits, and three mantissa bits with a bias value of 7. The design handled normal numbers, subnormal numbers, signed values, zero, infinity, and NaN cases according to the project requirements.

The adder was implemented by first separating each FP8 input into sign, exponent, and mantissa fields. Special cases such as NaN and infinity were checked before performing normal addition. For finite values, the inputs were converted into an integer representation where 512 units represent 1.0. The integer values were then added, and the result was converted back into FP8 format with rounding. If the result was outside the representable FP8 range, the output was set to infinity.

The design was verified using a Verilog testbench that tested all possible input combinations. Since each input is 8 bits, there were 256 possible values for input A and 256 possible values for input B, giving a total of 65,536 test cases. The simulation result showed zero errors, meaning the adder output matched the reference results for all input combinations.

The design was also implemented on the DE10-Lite FPGA board. Switches SW7–SW0 were used to enter FP8 values, KEY0 and KEY1 were used to store inputs A and B, and switches SW9–SW8 were used to select whether the displays show the current switch value, A, B, or the SUM. The six HEX displays showed the selected value in decimal floating-point form, including negative values, infinity, and NaN. FPGA tests confirmed correct operation for normal addition, negative results, NaN cases, infinity cases, and special-case rules.

Overall, the project successfully achieved the required FP8 floating-point addition, exhaustive simulation verification, and FPGA implementation. It also showed the limitation of FP8 precision, where some decimal results cannot be represented exactly and must be rounded to the nearest available FP8 value.